UCab – Website User Flows

Full-Stack Taxi Booking Application
Date: March 08, 2026

---

1. Overview

This document captures the end-to-end user flows for the UCab web application. It focuses on what the user does on the website (UI steps) and how the system responds at a high level, including real-time events with Socket.IO. Use this as a reference for design reviews, QA scenarios, and onboarding.

2. Actors & Assumptions

Actors: Rider, Driver (Admin features are future work).

Assumptions: Web UI built with React + Leaflet; backend Node/Express + MongoDB; JWT auth; Socket.IO for real-time ride requests and acceptance.

3. Global Pre- Conditions

- User has a stable internet connection and a modern browser.
- User account exists (or can self-register).
- Location permissions enabled for accurate pickup suggestions (optional).

4. Rider Flow (Website)

Register / Login

- Enter name, email/phone, password; role=rider.
- On success, JWT stored in memory/local storage (per app policy).

Set Pickup & Drop on Map

- Use map to select coordinates; auto-calculate distance and estimated fare.
- Validate field names: prefer pickupLocation / dropLocation; estimatedFare ↔ fare alignment.

Request Ride

- Create booking (status=pending). Rider joins rider-specific Socket room.
- Fallback UI if no drivers respond within SLA (e.g., 30–60s).

Wait for Acceptance (Real- time)

- Drivers in the matching vehicleType room receive newRideRequest.
- On first accept, rider receives rideAccepted; other drivers get rideRequestCancelled.

Track Ride

- Status phases: assigned → en-route → arrived → in-trip.
- Show driver details, ETA, and simulated/real position updates.

Complete, Rate & Pay

- Driver marks complete; rider rates driver; optional online payment.
- Show ride summary and allow raising a support ticket.

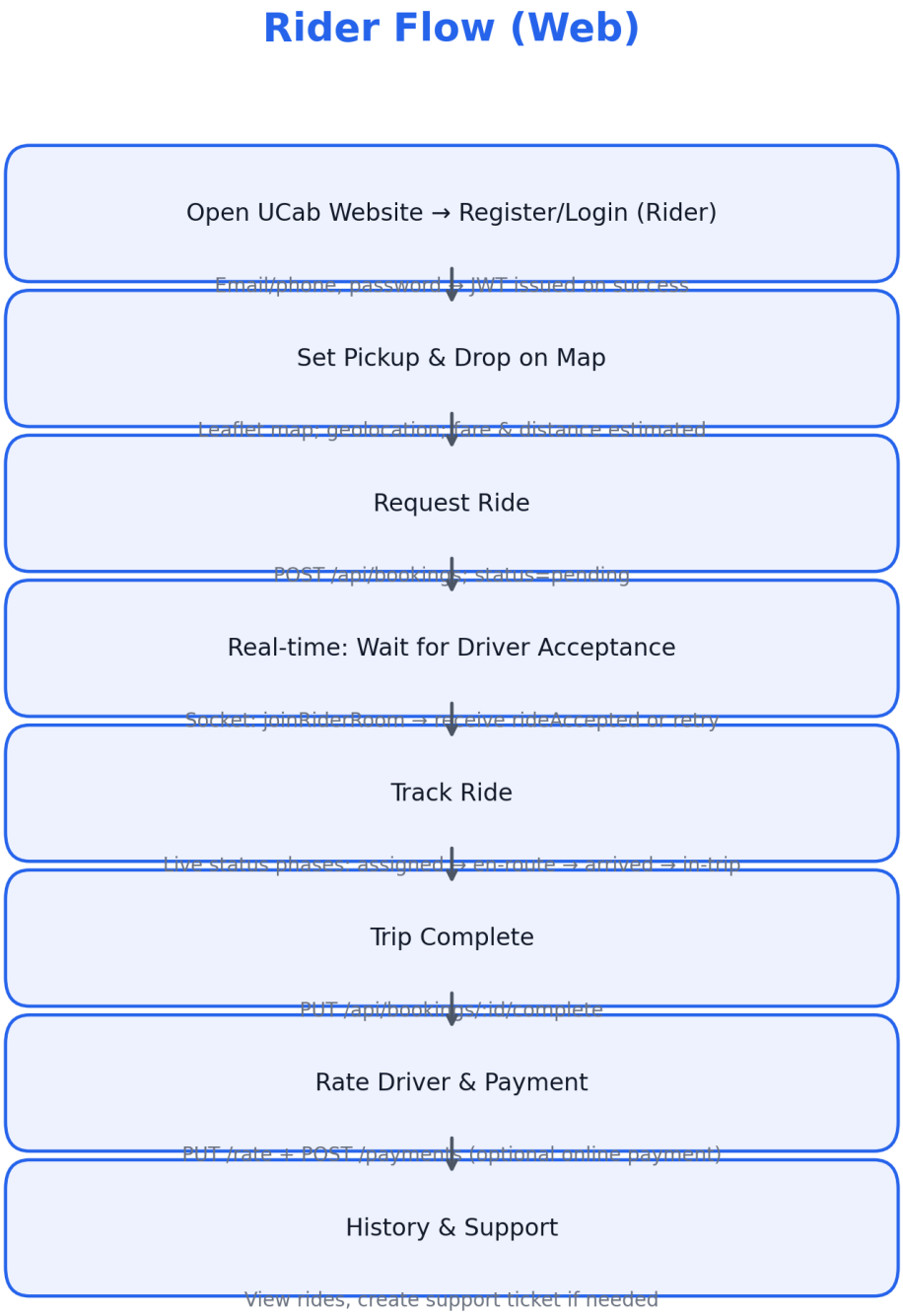

Figure 1: Rider flow on the website with key Socket.IO touchpoints.

5. Driver Flow (Website)

Login & Availability

- Login as driver; toggle availability in dashboard.
- Join vehicleType-specific Socket room (joinDriverRoom).

Receive & Accept Request

- Listen for newRideRequest; view pickup, distance, fare.
- Accept first; backend notifies rider and cancels others.

Navigate & Pickup

- Update ride status as progressing to pickup (assigned → en-route → arrived).
- Provide ETA updates to rider UI.

In- Trip & Complete

- Start trip (in-trip) and mark completed at drop location.
- Trip appears in driver history; rating averages update.

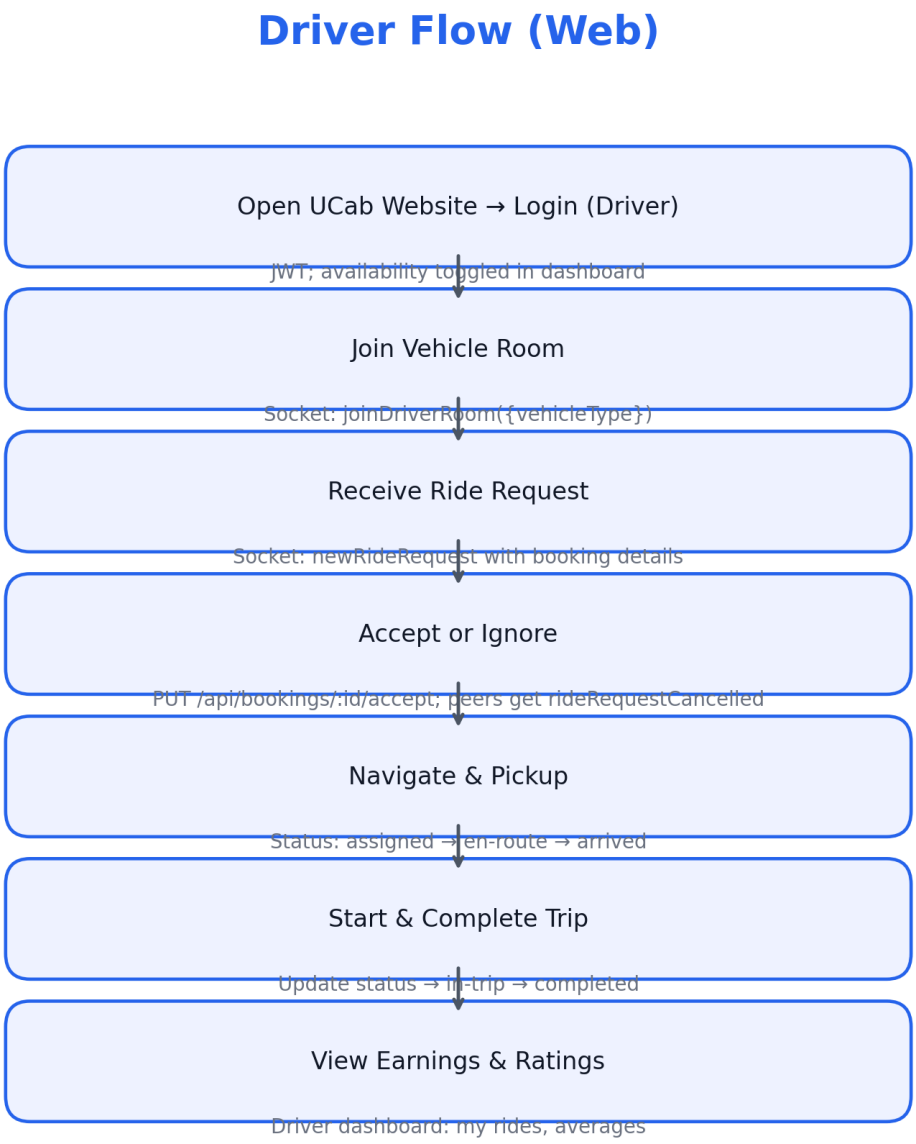

Figure 2: Driver flow on the website with acceptance and completion states.

6. Error States & Recovery

- Authentication failures → friendly error, retry, and forgot-password link.
- No drivers available → offer retry, change vehicle type, or schedule later.
- Socket disconnects → exponential backoff and visual offline banner.
- Backend failures → surface booking reference and support link.

7. Open Gaps Impacting Flows (from current implementation)

- Frontend calls on non-existent APIs (e.g., /api/bookings/search, /api/drivers/register, /api/drivers/:id/verify) should be removed or implemented to avoid dead-ends in UI.
- Field naming mismatches (dropoffLocation vs dropLocation, estimatedFare vs fare) must be aligned to prevent submission errors and broken summaries.
- Admin role implied by routes is not present in user schema; user flows referencing admin must remain hidden until role is formalized.
- Payment status is written to booking, but schema lacks the field; add it or remove UI references to payment state.

8. Appendix: Key Real- time Events & Statuses

Socket.IO Events

- Client → joinDriverRoom({ vehicleType })
- Client → joinRiderRoom({ riderId })
- Server → newRideRequest (to vehicleType room)
- Server → rideAccepted (to rider room)
- Server → rideRequestCancelled (notify competing drivers)

Booking Status Phases

- pending → assigned → en-route → arrived → in-trip → completed